

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills</i>		
Student Name:	AKTCHAYA A	Reg. No.:	192424096
Date:	03/08/2026	Duration:	60 minutes

Code of Conduct

I AKTCHAYA A certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:

Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. Study and Analyze the Scenario

- Carefully understand the given problem statement.
- Identify the business domain, stakeholders, existing challenges, and system objectives.

2. Identify Functional and Non-Functional Requirements

- List all functional requirements required by the proposed system.
- Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.

3. Identify Actors and Use Cases

- Identify all primary and secondary actors interacting with the system.
 - List the major use cases associated with each actor.
 - Draw a Use Case Diagram representing the interactions between actors and the system.
- 4. Prepare the Software Requirement Specification (SRS)**
- Develop an SRS document containing:
- Introduction
 - Problem Description
 - Scope of the System
 - Functional Requirements
 - Non-Functional Requirements
 - Use Case Descriptions
 - Data Requirements
 - External Interface Requirements
 - System Features
- 5. Identify Assumptions and Constraints**
- List the assumptions considered while developing the system.
 - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
- 6. Validate Requirement Completeness and Consistency**
- Verify whether all stakeholder requirements have been addressed.
 - Check for ambiguity, redundancy, inconsistency, and missing requirements.
- Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification	Clearly identifies comprehensive, accurate, and well classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.

Actor Identification and Use Case Modeling	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification	SRS is well structured, complete, professionally organized, and	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or

(SRS) Documentation	includes all required sections.			lacks essential components.
Assumptions, Constraints, and Requirement Validation	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	Mark
Scenario Understanding and Problem Analysis	/5
Functional and Non-Functional Requirement Identification	/5
Actor Identification and Use Case Modeling	/5
Software Requirement Specification (SRS) Documentation	/5
Assumptions, Constraints, and Requirement Validation	/5
TOTAL	/5

SCENARIO-BASED REQUIREMENT ANALYSIS

UNIVERSITY ACADEMIC MANAGEMENT AND STUDENT SUCCESS PLATFORM

1. Study and Analyze the Scenario

- **Business Domain**

Education Management System (Higher Education/University Academic Management).

- **Stakeholders**

- Students
- Faculty Members
- Academic Advisors
- Department Heads
- Examination Cell
- University Administrators
- Registrar
- Parents/Guardians (Optional)
- IT System Administrator

- **Existing Challenges**

- Student information is stored in multiple disconnected systems.
- Academic reporting is delayed due to manual processes.
- Communication between students and faculty is inefficient.
- Attendance tracking is time-consuming.
- Student performance monitoring is limited.
- Examination management lacks automation.
- Academic planning is based on incomplete data.
- Identifying at-risk students is difficult.
- Administrative workload is high.
- Institutional decision-making lacks real-time insights.

- **System Objectives**

- To centralize all academic management activities.
- To automate student admission and course registration.
- To monitor attendance and academic performance.
- To manage examinations and assessment records.
- To identify academically at-risk students.
- To notify students and faculty about academic updates.
- To generate institutional academic reports.
- To improve student engagement.
- To support academic planning and decision-making.
- To ensure secure and scalable university management.

2. Functional and Non-Functional Requirements

- **Functional Requirements**

- The system shall manage student admissions.
- The system shall maintain student academic records.
- The system shall support course registration.
- The system shall manage attendance records.
- The system shall record internal assessment marks.
- The system shall manage semester examinations.
- The system shall calculate student grades automatically.
- The system shall generate academic transcripts.
- The system shall monitor student academic progress.
- The system shall identify academically at-risk students.
- The system shall notify students about academic updates.
- The system shall notify faculty regarding student performance.
- The system shall allow faculty to upload marks.
- The system shall allow faculty to update attendance.
- The system shall generate departmental reports.
- The system shall generate institutional reports.
- The system shall manage faculty information.
- The system shall manage academic programs.
- The system shall provide dashboards for administrators.
- The system shall support multiple university departments.

- **Non-Functional Requirements**

- **Performance:** The system shall process user requests within three seconds.
- **Security:** The system shall provide role-based authentication and authorization.
- **Reliability:** The system shall maintain accurate academic records without data loss.
- **Usability:** The system shall provide a simple and user-friendly interface.
- **Scalability:** The system shall support thousands of students and faculty members.
- **Maintainability:** The system shall support easy software updates and maintenance.
- **Availability:** The system shall be available 24×7 with minimal downtime.
- **Portability:** The system shall support web and mobile platforms.
- **Backup and Recovery:** The system shall automatically back up academic data.
- **Compliance:** The system shall comply with university academic regulations.

3. Identify Actors and Use Cases

- **Primary Actors**
 - Student
 - Faculty Member
 - Academic Advisor
 - Administrator
 - Department Head
- **Secondary Actors**
 - Examination Cell
 - Registrar
 - IT Administrator
 - Parent/Guardian
- **Major Use Cases**
 - **Student**
 - Login
 - Register Courses
 - View Attendance
 - View Marks

- View Timetable
- Receive Notifications
- Download Academic Transcript
- **Faculty**
 - Login
 - Manage Attendance
 - Upload Marks
 - View Student Performance
 - Receive Student Alerts
- **Academic Advisor**
 - Monitor Student Progress
 - View At-Risk Students
 - Provide Academic Guidance
- **Administrator**
 - Manage Student Records
 - Manage Faculty Records
 - Manage Departments
 - Generate Reports
 - Manage Academic Policies
- **Department Head**
 - Monitor Department Performance
 - View Faculty Activities
 - Approve Academic Reports
- **Examination Cell**
 - Schedule Examinations
 - Publish Results
 - Generate Grade Sheets
- **IT Administrator**
 - Manage Users
 - Configure System
 - Backup Database
- **Use Case Diagram (Text Representation)**
 - **Student**
 - Login

- Register Courses
- View Attendance
- View Marks
- Receive Notifications
- **Faculty**
 - Login
 - Upload Marks
 - Update Attendance
 - View Student Progress
- **Academic Advisor**
 - Monitor Students
 - View At-Risk Students
- **Administrator**
 - Manage Students
 - Manage Faculty
 - Generate Reports
- **Department Head**
 - Monitor Department
- **Examination Cell**
 - Manage Examinations
- **IT Administrator**
 - Manage System

4. Software Requirement Specification (SRS)

- **Introduction**

The University Academic Management and Student Success Platform is a centralized software solution designed to automate academic administration, improve student success, and support institutional decision-making.

- **Problem Description**

Universities face challenges in managing academic records, attendance, examinations, and student performance due to fragmented systems and manual processes.

- **Scope of the System**

The system manages admissions, academic records, attendance, examinations, faculty activities, student performance, institutional reporting, and academic planning.

- **Functional Requirements**

- Student Admission Management
- Student Information Management
- Course Registration
- Attendance Management
- Assessment Management
- Examination Management
- Grade Calculation
- Academic Transcript Generation
- Student Performance Monitoring
- At-Risk Student Identification
- Notification Management
- Faculty Management
- Department Management
- Institutional Reporting
- Dashboard Management

- **Non-Functional Requirements**

- High Performance
- Strong Security
- High Reliability
- Easy Usability
- High Scalability
- Easy Maintainability
- High Availability
- Data Backup
- Cross-Platform Support
- Regulatory Compliance

- **Use Case Descriptions**

- **UC1 – Student Login:** Student securely logs into the system.
- **UC2 – Course Registration:** Student registers available academic courses.

- **UC3 – Attendance Management:** Faculty records and updates student attendance.
- **UC4 – Assessment Management:** Faculty uploads assessment marks.
- **UC5 – Examination Management:** Examination cell schedules and publishes examination results.
- **UC6 – Student Performance Monitoring:** System continuously evaluates student academic performance.
- **UC7 – At-Risk Student Identification:** System identifies students requiring academic intervention.
- **UC8 – Report Generation:** Administrator generates academic and institutional reports.
- **UC9 – Notification Management:** System sends academic alerts and notifications.
- **UC10 – Dashboard Monitoring:** Administrator monitors institutional performance through dashboards.
- **Data Requirements**
 - Student Information
 - Faculty Information
 - Department Information
 - Course Information
 - Attendance Records
 - Examination Records
 - Assessment Records
 - Grade Records
 - Academic Reports
 - Notification Records
 - User Login Details
 - Institutional Policies
- **External Interface Requirements**
 - **User Interface:** The system shall provide a responsive web-based dashboard.
 - **Hardware Interface:** The system shall support desktops, laptops, tablets, and mobile devices.
 - **Software Interface:** The system shall integrate with university databases and email services.

- **Communication Interface:** The system shall support internet-based communication using secure protocols.
- **System Features**
 - Student Admission
 - Course Registration
 - Attendance Tracking
 - Assessment Management
 - Examination Management
 - Grade Calculation
 - Student Performance Analytics
 - At-Risk Student Detection
 - Academic Notifications
 - Faculty Management
 - Department Management
 - Academic Reports
 - Institutional Dashboards
 - Secure User Authentication
 - Multi-Department Support

5. Assumptions and Constraints

- **Assumptions**
 - Students possess valid university registration details.
 - Faculty update attendance regularly.
 - Academic policies are predefined by the university.
 - Internet connectivity is available.
 - Users possess valid login credentials.
 - Examination schedules are maintained by the university.
 - Student information remains accurate.
 - Departments follow standardized academic procedures.
 - Institutional databases remain accessible.
 - Authorized users follow system usage policies.
- **Constraints**
 - **Technical Constraints**

- The system depends on stable internet connectivity.
- The system requires secure database storage.
- **Operational Constraints**
 - Faculty must update attendance on time.
 - Students must register within the academic schedule.
- **Economic Constraints**
 - The project must operate within the allocated university budget.
- **Legal Constraints**
 - Student information must comply with applicable data privacy regulations.
- **Time Constraints**
 - The project must be completed within the scheduled development timeline.

6. Validate Requirement Completeness and Consistency

- **Requirement Validation**
 - All stakeholder requirements are identified.
 - All functional requirements support the stated objectives.
 - All non-functional requirements are measurable.
 - All actors are linked with relevant use cases.
 - All academic processes are included in the system.
 - Data requirements support every system feature.
 - External interfaces satisfy user interaction needs.
- **Consistency Check**
 - No ambiguous requirements are present.
 - No duplicate requirements are included.
 - No conflicting requirements are identified.
 - All requirements follow consistent terminology.
 - Every requirement supports the project objectives.
- **Suggested Improvements**
 - Integrate AI-based student performance prediction.
 - Add mobile application support.
 - Enable cloud-based deployment.
 - Support learning management system integration.

- Include real-time analytics dashboards.
- Add chatbot-based student assistance.
- Implement automated timetable generation.
- Enable predictive academic intervention.
- Support online examination features.